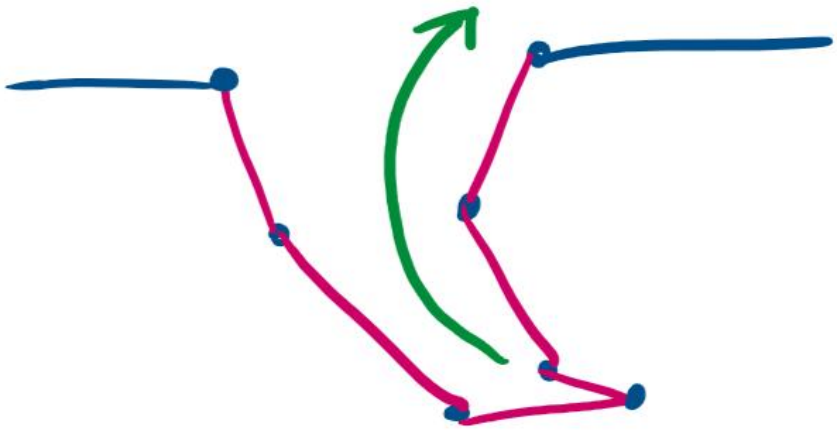
Problem 1

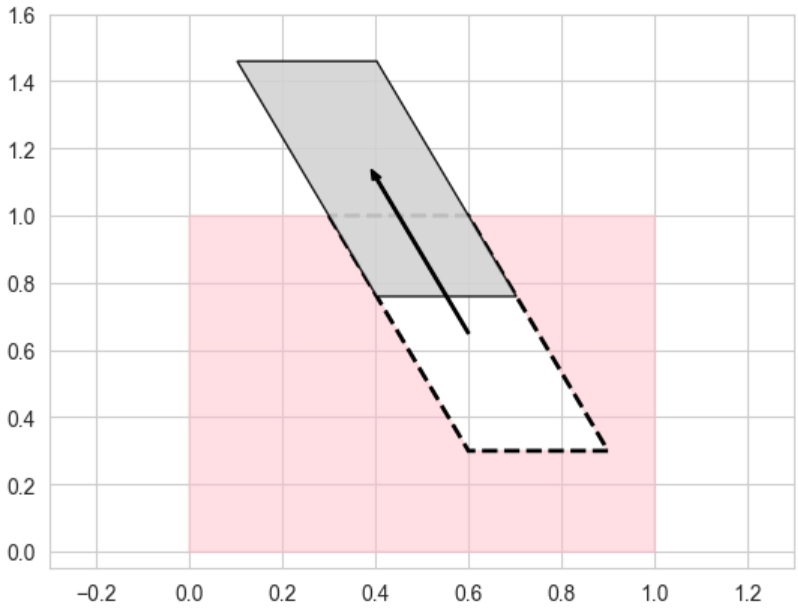
a)

Castable through rotation, but not translation:

Though not a polygon, one could imagine a crescent shape split exactly in the middle. This shape would be castable through rotation around the corresponding circle's center. To create a polygon example, we can just draw a 'low-resolution' version of the crescent.



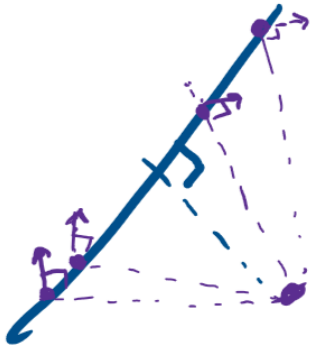
Castable through translation, but not rotation:



This shape is not rotatable due to the parallel nature of the edges of the parallelogram. No rotation is possible without driving an edge into the mold itself.

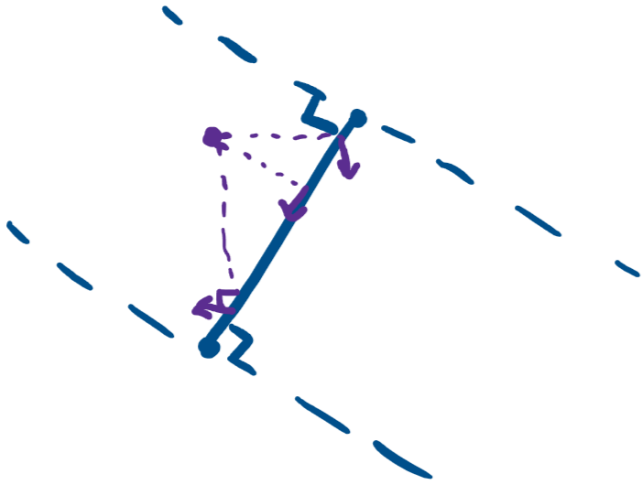
b) Finding a center of rotation for a castable polygon

Observe that for any rotation point, p, a line can be drawn from p to any edge on the polygon, let's call the edge E.



For any point *on* edge E, if rotated, will move perpendicular to the line drawn between the p_E and p. We will call this the differential movement. Due to the constraints of castability, the differential movement cannot go into the molds at any point on the edge E, as well as any point on the greater polygon.

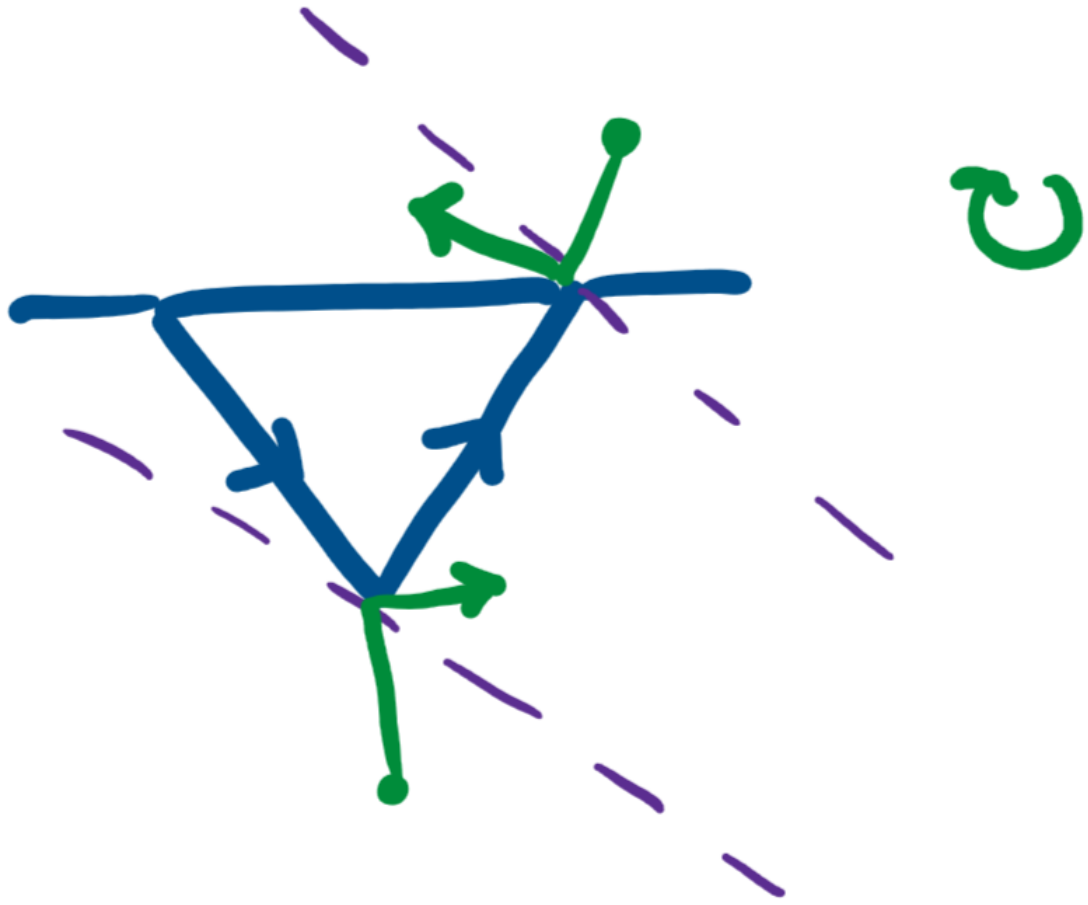
Knowing that the differential movement cannot go into the mold, we need to define a region that will only create the allowed differential movements, meaning all points that lie on edge E must move away from the mold.



Imagine that if p lies in between the slab describes by two parallel lines that are perpendicular to the edge E and the two parallel lines contain the respective end points of the edge E. Observe that within the slab, any point p within this slab will cause the differential motions of part of edge E to

go into the mold and the remaining part will exit the mold. Knowing this, we must remove the slab from the plane, leaving us with two half-planes.

To determine which of the half-planes we must choose, we need to consider the clockwise rotation and the greater context of the polygon. Assuming all points on the polygon are given in counterclockwise order, the half-plane 'pointed' to by the edge will always rotate the edge out of the mold; conversely, there is no point on the half-plane *not* 'pointed' to that will not rotate the edge into the mold.



This means the half-plane that is perpendicular to the edge E and starts at the later point of the edge (assuming counterclockwise orientation of the polygon), will always contain points that move the edge out of the mold. Any point on the respective half-plane for edge E will rotate E out of the mold. On a global scale, half-planes can be drawn for all edges on the polygon. If there is an intersection with all half-planes, then that means there must be a point p within that region that satisfies the rotatable constraints for all edges on the polygon, thus the polygon is rotatable.

Problem 2

See attached pdf of the Jupyter notebook.

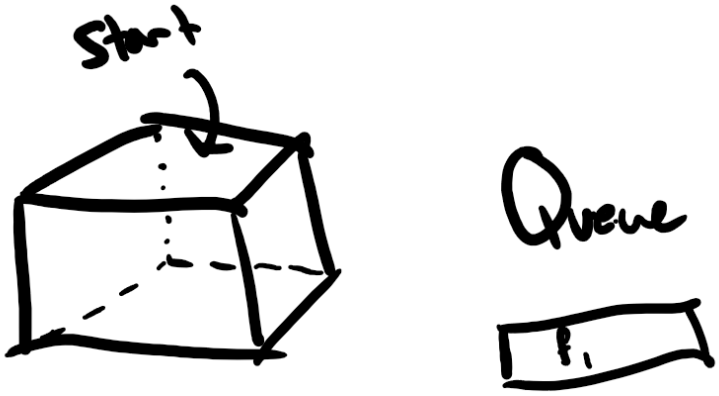
Problem 3

Overview and Setup

I assume the polyhedron stores its adjacent edges, vertices, and faces in a list. I assume that I can store whether or not the polygon is 'castable' through the face as a property of the face (store unknown, valid, or invalid) the polyhedron. We can alternatively store the polygon faces in a HashMap, storing the pointers to the faces as keys and the castability as a value.

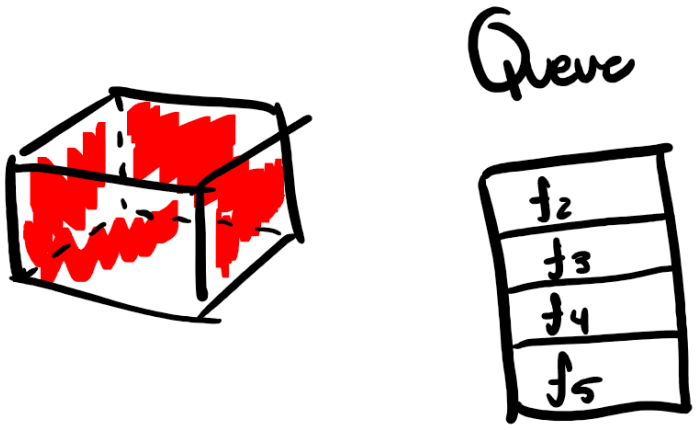
Algorithm

We can start by adding face, f_1, to a queue, Q.



For every face, f, in Q, we calculate the surface normal to f. We then calculate the dot product between the surface normal and the outgoing vector created from each of f's vertices. If *any* of the dot products are greater than 0, then mark f as invalid. Otherwise, f is valid.

Now, we add in all adjacent faces to f into the queue and remove f from the queue. If an adjacent face has already been checked, do not add it into the queue.



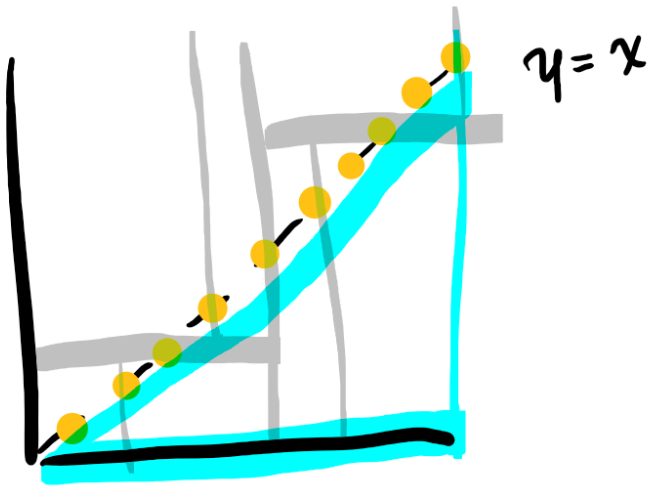
Analysis

Each face is only visited once; therefore, the algorithm will check each face exactly once. This yields O(n), where n represents the number of faces on the polyhedron.

Problem 5

a) Worst-Case is Ω(n)

As a proof by picture, we can see an example of a worst-case using data distributed across the function, y = x.



If our query range triangle has an hypotenuse that aligns exactly with the data along this line but only slightly below, the minimum run time is $\Omega(n)$ because the hypotenuse is able to touch every single sub-tree on the kd-tree, while none of the points actually fit the query.

b) Linear Data-Structure for Triangular Queries in $O(n^{3/4} + k)$

We can use a 4D-kd tree. For all coordinates, we can add the additional two dimensions by getting their coordinates using the basis vectors u = [1,1] and v = [1,-1]. In doing so, we get the benefits of the kd-tree and get the run-time of $O(n^{3/4} + k)$, due to the dimensional search range of a kd-tree scaling as $O(n^{1-1/d}+k)$ with d dimensions.

Problem 6

a) Single point query is O(log n) on a kd-tree

A search on a kd-tree for a single point means there is no additional k to the run-time. Leaving us with, $O(n^{1/2})$ as our run-time. But to further analyze, the gray nodes visited are only the parent nodes of the subtrees containing point (a,b). This means along the search to find (a,b) we only need to consider the depth of the kd-tree. The depth of the tree is ½ log(n) (defined in class), as such the total run-time is O(log n).

b) Single point query on a range-tree

A defined in part a), the search reduces to the depth needed to traverse in order to reach the singular point. Due to the nature of a range-tree, in the worst-case scenario, the x-range tree and the y-range tree must be search in their entirety. In this scenario, the height of each tree log(n), so the combined height is 2 log(n). As such, the total run-time is O(log n).